

Food Delivery *System*



Análise e Modelagem Orientada a Objetos de uma
Plataforma de Delivery com 500+ Restaurantes
Parceiros

Documento técnico detalhado cobrindo requisitos funcionais e não funcionais, diagramas UML de casos de uso, classes, objetos, pacotes e sequência, além de exemplo de implementação em Java. Modelo de referência para desenvolvimento mobile (Android/iOS) e web admin.

AUTOR

Engenharia de Software

VERSÃO

1.0

DATA

14 de Agosto de 2026

Sumário

1. Introdução e Contexto do Projeto	4
1.1 Visão Geral do Sistema	4
1.2 Objetivos do Projeto	4
1.3 Escopo	4
1.4 Público-Alvo	5
1.5 Plataforma e Stack Tecnológica	5
2. Levantamento de Requisitos	6
2.1 Requisitos Funcionais (RF)	6
2.2 Requisitos Não Funcionais (RNF)	7
2.3 Matriz de Priorização	8
3. Diagrama de Casos de Uso	9
3.1 Atores do Sistema	9
3.2 Diagrama de Casos de Uso	9
3.3 Descrição Textual dos Casos de Uso Principais	10
UC-01: Fazer Pedido	10
UC-02: Rastrear Pedido	11
UC-03: Aceitar Entrega	12
UC-04: Gerenciar Cardápio	12
UC-05: Processar Pagamento	13
4. Diagrama de Classes	13
4.1 Visão Geral das Classes	14
4.2 Diagrama de Classes	14

4.3 Descrição das Classes	14
4.4 Relacionamentos entre Classes	15
5. Diagrama de Objetos	16
5.1 Cenário Modelado	16
5.2 Diagrama de Objetos	17
5.3 Descrição das Instâncias	17
6. Diagrama de Pacotes	18
6.1 Visão Arquitetural	18
6.2 Diagrama de Pacotes	18
6.3 Descrição dos Pacotes	19
7. Diagrama de Sequência	20
7.1 Cenário: Cliente Realiza Pedido com Sucesso	20
7.2 Diagrama de Sequência	20
7.3 Descrição Passo-a-Passo do Fluxo	21
8. Exemplo de Implementação em Java	22
8.1 Classe Pedido	22
8.2 Classe Entregador	23
8.3 Mapeamento UML → Código	24
9. Tabela Resumo de Relações UML	25
10. Diferenciais do Sistema	26
10.1 Funcionalidades Mobile (Android/iOS)	26
10.2 Funcionalidades Web Admin	26
10.3 Integrações com Serviços Externos	27

11. Considerações Finais

28

1. Introdução e Contexto do Projeto

1.1 Visão Geral do Sistema

Este documento apresenta o projeto completo de modelagem UML de uma plataforma de **delivery de comida** (food delivery) desenvolvida para uma startup em fase de crescimento que já conta com mais de 500 restaurantes parceiros cadastrados. A modelagem foi conduzida segundo as boas práticas de Engenharia de Software e segundo a notação UML 2.5 (ISO/IEC 19505), cobrindo desde o levantamento de requisitos até a especificação de classes, objetos, pacotes e fluxos de interação entre componentes.

O sistema modelado atende a quatro perfis distintos de usuários: **clientes** que realizam pedidos e acompanham entregas em tempo real; **restaurantes** parceiros que gerenciam cardápios, aceitam pedidos e criam promoções; **entregadores** autônomos que recebem notificações de entrega e atualizam status; e **administradores** da plataforma responsáveis por métricas, moderação e gestão financeira. A arquitetura prevê ainda integração com **atores externos** como gateways de pagamento e serviços de mapeamento.

A escolha por uma modelagem UML completa — em vez de especificações informais — justifica-se pela complexidade do domínio: transações financeiras, geolocalização em tempo real, comunicação assíncrona entre atores e múltiplas regras de negócio (cupons, reembolsos, comissões). Diagramas formais reduzem ambiguidades na comunicação entre stakeholders técnicos e não técnicos, além de servirem como contrato de implementação para a equipe de desenvolvimento.

1.2 Objetivos do Projeto

O projeto tem por objetivos: (i) **mapear o domínio funcional** do sistema de delivery, identificando atores, casos de uso e seus relacionamentos; (ii) **estruturar o modelo de classes** com responsabilidades claras e baixo acoplamento, seguindo princípios SOLID; (iii) **documentar a arquitetura em camadas** por meio do diagrama de pacotes, separando apresentação, negócio, dados e integrações; (iv) **exemplificar o comportamento dinâmico** do sistema através de diagramas de sequência e objetos; e (v) **fornecer um exemplo de implementação** em Java que demonstre como relacionamentos UML como <> e <> se traduzem em código executável.

1.3 Escopo

O escopo cobre as funcionalidades essenciais de um aplicativo de delivery moderno: autenticação de usuários, catálogo de restaurantes com busca e filtros, carrinho de compras, checkout com múltiplos métodos de pagamento, rastreamento em tempo real, avaliações, gestão de cardápio, atribuição de entregas, cupons de desconto, chat

cliente-entregador, histórico de pedidos, cancelamentos e reembolsos, gestão de múltiplos endereços, notificações push e painel administrativo com métricas.

Estão fora do escopo deste documento: detalhes de implementação de infraestrutura (Kubernetes, CI/CD), contratos de API REST específicos, modelagem de banco de dados relacional (DER), plano de testes e estratégia de monitoramento em produção. Esses artefatos seriam produzidos em fases subsequentes do ciclo de desenvolvimento.

1.4 Público-Alvo

O sistema atende a quatro perfis principais de usuários, cada um com fluxos e permissões distintos. A tabela abaixo resume os perfis, suas responsabilidades e canais de acesso preferenciais.

Perfil	Responsabilidades	Canal de Acesso
Cliente	Realizar pedidos, pagar, rastrear entrega, avaliar restaurantes e entregadores, gerenciar endereços e cupons	App Mobile (iOS/Android)
Restaurante	Gerenciar cardápio, aceitar/rejeitar pedidos, criar promoções, acompanhar métricas de vendas	Web App / Tablet
Entregador	Receber notificações de entrega, aceitar/recusar, navegar via GPS, atualizar status, confirmar entrega	App Mobile (Android)
Administrador	Gerenciar usuários, aprovar restaurantes, monitorar transações, gerar relatórios, resolver conflitos	Web Admin (Desktop)
Sistema de Pagamento	Ator externo: processa transações via cartão, PIX e carteira digital; retorna confirmação ou falha	API REST (gateway)

Tabela 1.1 — Perfis de usuários e atores do sistema

1.5 Plataforma e Stack Tecnológica

A plataforma é composta por três aplicativos móveis distintos (cliente, entregador e restaurante), um painel web administrativo e serviços de backend. A stack recomendada para implementação inclui tecnologias consolidadas no mercado, conforme detalhado na seção de diferenciais do sistema.

A camada mobile pode ser desenvolvida em React Native ou Flutter para reduzir o esforço de manutenção multiplataforma. O backend deve seguir arquitetura de microsserviços com API Gateway, utilizando Node.js + TypeScript ou Spring Boot (Java/Kotlin) para serviços de domínio. A persistência utiliza PostgreSQL para dados transacionais, Redis para cache e filas, e MongoDB opcional para logs de auditoria.

2. Levantamento de Requisitos

O levantamento de requisitos é a fase fundamental que define **o que** o sistema deve fazer (Requisitos Funcionais — RF) e **como** deve se comportar em termos de qualidade (Requisitos Não Funcionais — RNF). Esta seção apresenta 15 requisitos funcionais e 12 requisitos não funcionais, organizados por categoria e prioridade. A priorização segue a técnica MoSCoW (Must, Should, Could), simplificada em Alta/Média/Baixa para fins didáticos.

2.1 Requisitos Funcionais (RF)

Requisitos funcionais descrevem as funcionalidades específicas que o sistema deve oferecer aos usuários. Cada requisito possui um identificador único (RF-XX), descrição, categoria e prioridade. A prioridade **Alta** indica funcionalidades essenciais para o MVP; **Média** indica recursos importantes para lançamento completo; **Baixa** indica melhorias incrementais pós-lançamento.

ID	Descrição	Categoria	Prior.
RF-01	Cadastro e autenticação de usuários (clientes, entregadores e restaurantes) com validação de e-mail, CPF/CNPJ e telefone	Acesso	Alta
RF-02	Busca e filtro de restaurantes por categoria, localização (raio em km), avaliação média e tempo de entrega	Catálogo	Alta
RF-03	Realização de pedidos com carrinho de compras: adicionar/remover itens, definir quantidade, observações por item	Pedido	Alta
RF-04	Pagamento online integrado com cartão de crédito, PIX e carteira digital, com confirmação assíncrona via webhook	Pagamento	Alta
RF-05	Rastreamento de pedido em tempo real com atualização de localização do entregador a cada 5 segundos via WebSocket	Rastreamento	Alta
RF-06	Avaliação de restaurantes e entregadores com nota de 1 a 5 estrelas, comentário opcional e moderação administrativa	Avaliação	Média
RF-07	Gestão de cardápio pelos restaurantes: cadastrar produtos com foto, descrição, preço, categoria e disponibilidade	Cardápio	Alta
RF-08	Atribuição automática de entregas aos entregadores disponíveis com base em localização, veículo e algoritmo de matching	Logística	Alta

ID	Descrição	Categoria	Prior.
RF-09	Gestão de promoções e cupons de desconto: percentual, fixo, frete grátis, validade, limite de usos e segmentação por cliente	Marketing	Média
RF-10	Chat em tempo real entre cliente e entregador durante o ciclo de entrega, com histórico persistente por 30 dias	Comunicação	Média
RF-11	Histórico de pedidos com filtros por período, status e restaurante, permitindo refazer pedidos com um toque	Pedido	Média
RF-12	Cancelamento e reembolso de pedidos com regras de negócio por status, prazo e motivo, integrado ao gateway de pagamento	Pedido	Alta
RF-13	Gestão de múltiplos endereços de entrega por cliente com geolocalização via Google Maps e validação de CEP	Endereço	Média
RF-14	Notificações push de status do pedido (criado, aceito, em preparo, saiu para entrega, entregue) via Firebase Cloud Messaging	Notificação	Alta
RF-15	Painel administrativo com métricas em tempo real: GMV, ticket médio, restaurantes ativos, NPS e relatórios exportáveis (CSV/PDF)	Admin	Média

Tabela 2.1 — Requisitos Funcionais do Sistema (RF-01 a RF-15)

2.2 Requisitos Não Funcionais (RNF)

Requisitos não funcionais especificam os atributos de qualidade que o sistema deve possuir: performance, escalabilidade, disponibilidade, segurança, conformidade legal, usabilidade, entre outros. Estes requisitos são frequentemente mais críticos que os funcionais, pois determinam a viabilidade operacional e a experiência do usuário em escala. Cada RNF deve ser mensurável e testável.

ID	Categoria	Especificação Mensurável	Métrica
RNF-01	Performance	Tempo de resposta das APIs críticas (busca, pedido, pagamento) inferior a 1,5 segundos no percentil 95	p95 < 1.500ms
RNF-02	Escalabilidade	Suportar até 10.000 pedidos simultâneos com autoescalando horizontal dos microsserviços	10k ops/s
RNF-03	Disponibilidade	Uptime de 99,95% (máximo de 4,38 horas de downtime por ano) com failover automático entre regiões	99,95% uptime

ID	Categoria	Especificação Mensurável	Métrica
RNF-04	Segurança	Criptografia AES-256 em repouso para dados sensíveis, TLS 1.3 em trânsito, senhas com bcrypt (cost 12)	AES-256 / TLS 1.3
RNF-05	Conformidade	Conformidade total com LGPD: consentimento explícito, direito ao esquecimento, registro de operações com dados pessoais	LGPD 100%
RNF-06	Usabilidade	Interface intuitiva com no máximo 3 toques para concluir um pedido a partir da tela inicial	≤ 3 toques
RNF-07	Compatibilidade	Suporte a Android 8.0+ (API 26+) e iOS 13.0+, cobrindo 95% dos dispositivos ativos no mercado brasileiro	Android 8+ / iOS 13+
RNF-08	Geolocalização	Precisão de localização inferior a 10 metros em ambiente urbano, utilizando GPS assistido por Wi-Fi e torres celulares	≤ 10m
RNF-09	Integração	Integração com gateways de pagamento (Stripe, Mercado Pago) via API REST com idempotência e retry exponencial	99,9% success
RNF-10	Backup	Backup automatizado a cada 6 horas com RPO de 15 minutos e RTO inferior a 30 minutos para recuperação de desastres	RTO < 30min
RNF-11	Acessibilidade	Suporte a leitores de tela (TalkBack/VoiceOver), contraste WCAG AA, suporte a fontes ampliadas até 200%	WCAG 2.1 AA
RNF-12	Internacionalização	Arquitetura i18n com suporte inicial a Português, Inglês e Espanhol, com tradução de UI, moeda e fuso horário	3 idiomas

Tabela 2.2 — Requisitos Não Funcionais do Sistema (RNF-01 a RNF-12)

2.3 Matriz de Priorização

A matriz abaixo consolida a distribuição dos requisitos por prioridade e categoria. Observa-se que 8 dos 15 requisitos funcionais (53%) são classificados como prioridade Alta, refletindo o foco do MVP em funcionalidades essenciais de pedido, pagamento e rastreamento. Os requisitos de prioridade Baixa não foram incluídos pois dependem de validação de mercado pós-MVP.

Prioridade	Quantidade RF	Percentual	Fase de Entrega
Alta	8	53%	MVP — 3 meses
Média	7	47%	V1.0 — 6 meses
Baixa	0	0%	Pós-validação

Tabela 2.3 — Matriz de priorização dos requisitos funcionais

3. Diagrama de Casos de Uso

O diagrama de casos de uso captura a visão externa do sistema, descrevendo as interações entre atores (humanos ou sistemas externos) e as funcionalidades que o sistema oferece. É o diagrama mais utilizado para comunicação com stakeholders não técnicos, pois não exige conhecimento de programação. A notação empregada segue a UML 2.5 e inclui os estereótipos <> (comportamento obrigatório compartilhado) e <> (comportamento opcional condicional).

3.1 Atores do Sistema

Foram identificados cinco atores no sistema, sendo quatro primários (que iniciam interações) e um secundário (ator externo que responde a chamadas do sistema). Atores primários são representados por figuras humanóides padrão; atores externos mantêm a mesma notação mas recebem rótulo <> quando necessário.

Ator	Tipo	Descrição
Cliente	Primário	Usuário final que realiza pedidos, efetua pagamentos, rastreia entregas, avalia estabelecimentos e gerencia seu perfil e endereços
Restaurante	Primário	Estabelecimento parceiro que gerencia cardápio, aceita ou rejeita pedidos, cria promoções e acompanha métricas de venda
Entregador	Primário	Parceiro autônomo que recebe solicitações de entrega, navega até o restaurante e o cliente, atualiza status e confirma entrega
Administrador	Primário	Funcionário da startup que opera o painel administrativo, gerencia usuários, aprova restaurantes, monitora transações e gera relatórios
Sistema de Pagamento	Externo	Gateway de pagamento externo (Stripe, Mercado Pago) que processa transações financeiras e retorna confirmações via webhook

Tabela 3.1 — Atores identificados no sistema

3.2 Diagrama de Casos de Uso

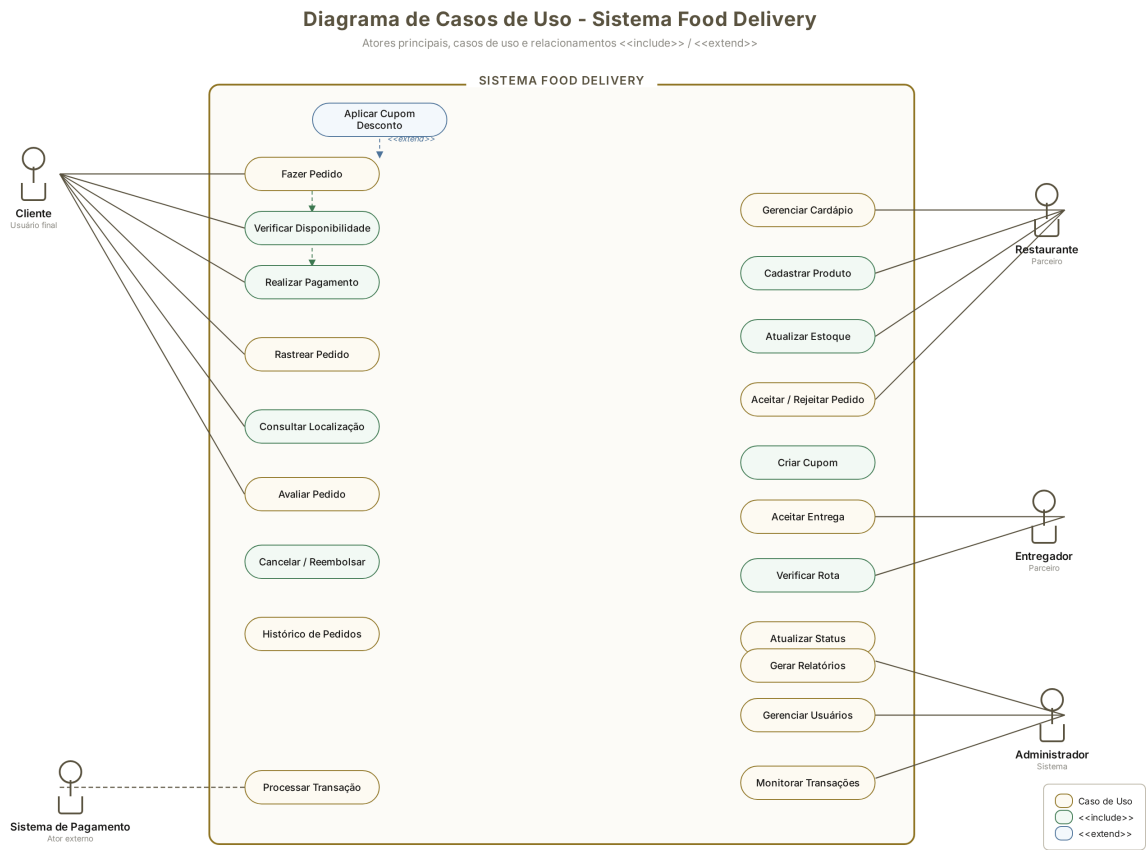


Figura 3.1 — Diagrama de casos de uso do Sistema Food Delivery

3.3 Descrição Textual dos Casos de Uso Principais

A seguir são detalhados os cinco casos de uso mais críticos do sistema, em formato estruturado com ator, pré-condição, fluxo principal, fluxos alternativos, pós-condição e exceções. Esta especificação é essencial para que desenvolvedores e QA entendam o comportamento esperado sem ambiguidades.

UC-01: Fazer Pedido

Aspecto	Descrição
Ator Principal	Cliente
Pré-condição	Cliente está autenticado no aplicativo e possui ao menos um endereço de entrega cadastrado

Aspecto	Descrição
Fluxo Principal	1. Cliente busca restaurante ou seleciona da lista 2. Sistema exibe cardápio do restaurante 3. Cliente adiciona itens ao carrinho com quantidade e observações 4. Cliente revisa o carrinho e valor total 5. Cliente seleciona endereço de entrega 6. <> Sistema verifica disponibilidade dos itens 7. Cliente seleciona método de pagamento 8. <> Sistema processa pagamento via gateway externo 9. <> Cliente aplica cupom de desconto (opcional) 10. Cliente confirma pedido 11. Sistema cria pedido e notifica restaurante
Fluxos Alternativos	A1: Item indisponível — Sistema sugere substituto ou remove do carrinho A2: Pagamento recusado — Sistema oferece novo método A3: Cliente agenda entrega para horário futuro (<> Agendar Entrega)
Pós-condição	Pedido criado com status "Aguardando confirmação do restaurante" e notificação enviada
Exceções	Restaurante fechado durante checkout; falha de comunicação com gateway; carrinho vazio

Tabela 3.2 — Especificação do caso de uso Fazer Pedido

UC-02: Rastrear Pedido

Aspecto	Descrição
Ator Principal	Cliente
Pré-condição	Cliente possui pedido ativo com status entre "Confirmado" e "Em entrega"
Fluxo Principal	1. Cliente acessa tela "Meus Pedidos" 2. Sistema lista pedidos ativos 3. Cliente seleciona pedido para rastreamento 4. <> Sistema consulta localização atual do entregador 5. Sistema exibe mapa com posição do entregador, rota e ETA 6. Sistema atualiza posição a cada 5 segundos via WebSocket 7. Cliente visualiza status em tempo real até "Entregue"
Fluxos Alternativos	A1: Pedido ainda em preparo — Sistema mostra status do restaurante A2: Conexão WebSocket cai — App faz polling a cada 15 segundos como fallback
Pós-condição	Cliente visualiza status atualizado até a entrega ser confirmada
Exceções	GPS do entregador indisponível; pedido cancelado durante rastreamento

Tabela 3.3 — Especificação do caso de uso Rastrear Pedido

UC-03: Aceitar Entrega

Aspecto	Descrição
Ator Principal	Entregador
Pré-condição	Entregador está autenticado, online e com status "disponível" no app
Fluxo Principal	1. Sistema identifica novo pedido pronto para coleta próximo ao entregador 2. Sistema envia notificação push com detalhes (restaurante, endereço, valor) 3. Entregador visualiza oferta e toca em "Aceitar" 4. <> Sistema verifica rota ótima até o restaurante e o cliente 5. Sistema confirma atribuição e bloqueia o pedido para outros entregadores 6. Entregador navega até o restaurante para coleta 7. Entregador atualiza status para "Em rota de entrega"
Fluxos Alternativos	A1: Entregador recusa — Sistema oferta ao próximo entregador disponível A2: Nenhum entregador aceita em 5 minutos — Pedido entra em fila prioritária
Pós-condição	Pedido atribuído ao entregador, status atualizado para "Saiu para entrega"
Exceções	Entregador perde conexão; rota inviável por evento externo; <> Reportar Problema

Tabela 3.4 — Especificação do caso de uso Aceitar Entrega

UC-04: Gerenciar Cardápio

Aspecto	Descrição
Ator Principal	Restaurante
Pré-condição	Restaurante autenticado no painel administrativo e previamente aprovado pelo Administrador
Fluxo Principal	1. Restaurante acessa aba "Cardápio" 2. <> Sistema lista produtos cadastrados com status (ativo/inativo) 3. Restaurante pode: cadastrar novo produto, editar existente ou remover 4. <> Cadastrar Produto: nome, descrição, preço, categoria, foto, tempo de preparo 5. <> Atualizar Estoque: marcar como disponível/indisponível 6. Sistema valida campos e salva alterações 7. Sistema atualiza catálogo em tempo real para clientes visualizando
Fluxos Alternativos	A1: Preço inválido — Sistema rejeita e exibe mensagem A2: Foto com formato não suportado — Sistema solicita nova upload
Pós-condição	Cardápio do restaurante atualizado e visível para clientes em até 30 segundos

Aspecto	Descrição
Exceções	Upload de imagem falha; limite de produtos excedido; campos obrigatórios em branco

Tabela 3.5 — Especificação do caso de uso Gerenciar Cardápio

UC-05: Processar Pagamento

Aspecto	Descrição
Ator Principal	Sistema (interna), com Sistema de Pagamento como ator externo
Pré-condição	Pedido criado com valor total calculado e itens confirmados como disponíveis
Fluxo Principal	1. Sistema recebe solicitação de pagamento do App Mobile 2. Sistema valida dados do pagamento (método, valor, cliente) 3. Sistema gera ID interno de transação para idempotência 4. Sistema envia requisição ao Gateway de Pagamento externo (Stripe/Mercado Pago) 5. Gateway valida fundos e processa transação 6. Gateway retorna confirmação com transactionId externo 7. Sistema atualiza status do Pagamento para "Aprovado" 8. Sistema libera pedido para o fluxo de preparação 9. Sistema envia notificação ao cliente
Fluxos Alternativos	A1: Gateway retorna "saldo insuficiente" — Sistema marca Pagamento como "Recusado" e notifica cliente A2: Timeout do gateway — Sistema faz retry com backoff exponencial (3 tentativas) A3: Webhook assíncrono recebido antes do timeout — Sistema reconcilia estados
Pós-condição	Pagamento processado com sucesso ou registrado como falha para retry/manual
Exceções	Indisponibilidade do gateway; dados de cartão inválidos; duplicação por retry mal implementado

Tabela 3.6 — Especificação do caso de uso Processar Pagamento

4. Diagrama de Classes

O diagrama de classes é o núcleo da modelagem orientada a objetos. Ele descreve a estrutura estática do sistema: classes, atributos, métodos e relacionamentos. Diferente dos casos de uso (visão externa) e da sequência (visão dinâmica), o diagrama de classes captura o **modelo de domínio** — as entidades que existem no problema e como elas se conectam. Esta seção apresenta dez classes principais do sistema de delivery, com herança, associação, agregação e composição.

4.1 Visão Geral das Classes

A modelagem utiliza uma hierarquia com **Usuario** como classe abstrata base, especializada em **Cliente** e **Entregador**. Esta abstração evita duplicação de atributos comuns (id, nome, email, telefone, senha) e centraliza operações de autenticação. A classe **Pedido** atua como agregadora central, conectando-se a **Cliente**, **Restaurante**, **Entregador**, **Pagamento**, **Avaliacao** e compondo múltiplos **ItemPedido**. A separação entre **ItemPedido** e **Produto** é intencional: o ItemPedido registra o estado do produto no momento da compra (preço, observações), preservando o histórico mesmo se o Produto for posteriormente alterado.

4.2 Diagrama de Classes

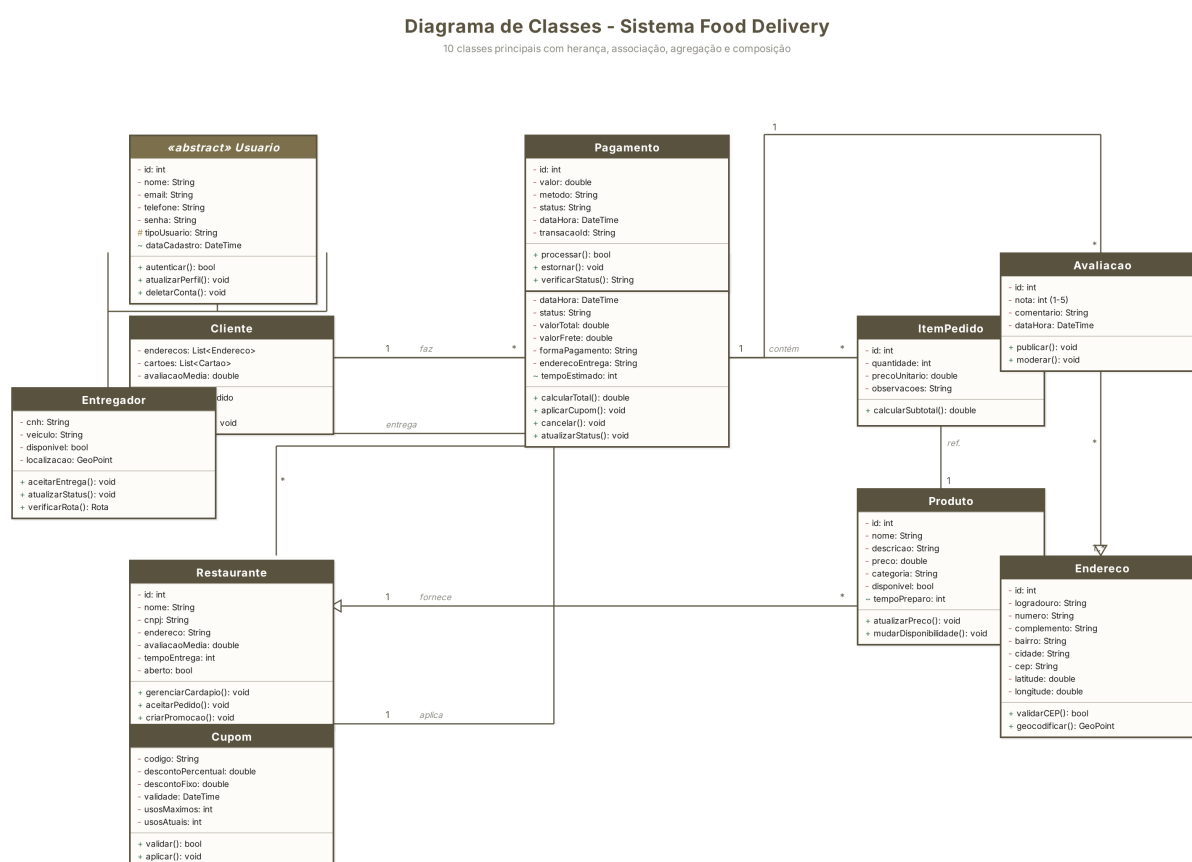


Figura 4.1 — Diagrama de classes do Sistema Food Delivery

4.3 Descrição das Classes

Classe	Tipo	Responsabilidade
Usuario	Abstract	Superclasse abstrata com atributos comuns a todos os usuários (id, nome, email, senha) e operações de autenticação e gestão de conta

Classe	Tipo	Responsabilidade
Cliente	Concreta	Especialização de Usuario que mantém endereços e cartões; executa fazerPedido(), avaliar() e cancelarPedido()
Entregador	Concreta	Especialização de Usuario com CNH, veículo e disponibilidade; métodos aceitarEntrega(), atualizarStatus(), verificarRota()
Restaurante	Concreta	Estabelecimento parceiro com CNPJ, avaliação média e tempo de entrega; gerencia cardápio, aceita pedidos e cria promoções
Pedido	Concreta	Classe central que agrega todas as informações de uma transação: itens, cliente, restaurante, entregador, pagamento, status e valores
ItemPedido	Concreta	Composição de Pedido: representa um item específico dentro de um pedido, com quantidade, preço unitário e observações
Produto	Concreta	Item do cardápio do restaurante; agregado por Restaurante; referenciado por ItemPedido para manter preço histórico no pedido
Pagamento	Concreta	Registra a transação financeira: valor, método (cartão/PIX/carteira), status, transacaoId do gateway e operações de processar/estornar
Avaliacao	Concreta	Nota (1-5) e comentário deixado pelo cliente sobre um pedido; pode ser moderada pelo administrador
Cupom	Concreta	Código promocional com desconto percentual ou fixo, validade e limites de uso; validado e aplicado a um Pedido
Endereco	Concreta	Localização geográfica com logradouro, CEP e coordenadas; agregado por Cliente e usado em Pedido como destino de entrega

Tabela 4.1 — Descrição das classes do modelo de domínio

4.4 Relacionamentos entre Classes

Os relacionamentos definem como as classes se conectam e colaboram. A correta escolha entre associação, agregação e composição impacta diretamente o ciclo de vida dos objetos e a integridade dos dados. A tabela abaixo detalha cada relacionamento, sua multiplicidade e justificativa.

Relacionamento	Tipo	Multiplicidade	Justificativa
Cliente — Pedido	Associação	1 : *	Um cliente pode fazer múltiplos pedidos ao longo do tempo; cada pedido pertence a um único cliente
Pedido — ItemPedido	Composição	1 : *	ItemPedido não existe sem o Pedido; se o pedido é cancelado, seus itens são removidos em cascata

Relacionamento	Tipo	Multiplicidade	Justificativa
ItemPedido — Produto	Associação	* : 1	Vários itens podem referenciar o mesmo produto; o produto persiste mesmo após pedidos serem concluídos
Restaurante — Produto	Agregação	1 : *	Restaurante agrega produtos como parte de seu cardápio; produtos podem existir teoricamente sem restaurante (catálogo genérico)
Pedido — Pagamento	Associação	1 : 1	Cada pedido possui exatamente um pagamento associado; o pagamento pode ter múltiplas tentativas registradas como histórico
Pedido — Avaliacao	Associação	1 : 0..1	Um pedido pode ou não receber avaliação; a avaliação é opcional e única por pedido
Entregador — Pedido	Associação	1 : *	Um entregador realiza múltiplos pedidos ao longo do tempo; cada pedido é atribuído a um entregador por vez
Pedido — Restaurante	Associação	* : 1	Múltiplos pedidos podem ser feitos no mesmo restaurante; cada pedido é de um único restaurante
Cliente — Endereco	Agregação	1 : 1..*	Cliente agrega um ou mais endereços; endereços podem existir independentemente em catálogo de CEP
Usuario ← Cliente	Herança	— is a —	Cliente é um tipo de Usuario; herda todos os atributos e métodos da superclasse abstrata

Tabela 4.2 — Relacionamentos entre classes do modelo

5. Diagrama de Objetos

Enquanto o diagrama de classes mostra a estrutura abstrata do modelo, o diagrama de objetos apresenta uma **fotografia** do sistema em um instante específico do tempo, com instâncias concretas das classes e valores reais de atributos. Este diagrama é particularmente útil para validar o modelo de classes, comunicar exemplos de uso reais e servir de base para testes de integração. A UML permite representar tanto objetos individuais quanto os **links** (instâncias de associações) entre eles.

5.1 Cenário Modelado

O cenário escolhido retrata o momento em que a cliente **Maria Santos** realiza o pedido #1001 no restaurante **Burguer House**, pagando via PIX. O pedido contém 2 unidades do

X-Burger Especial (com observação 'sem cebola') e está em status 'Em preparação'. O pagamento foi aprovado (transação PIX-20240325-889), e o entregador **João Pereira** já está designado para a entrega, aguardando o restaurante finalizar o preparo. Este snapshot permite validar todos os relacionamentos do diagrama de classes em um caso concreto.

5.2 Diagrama de Objetos

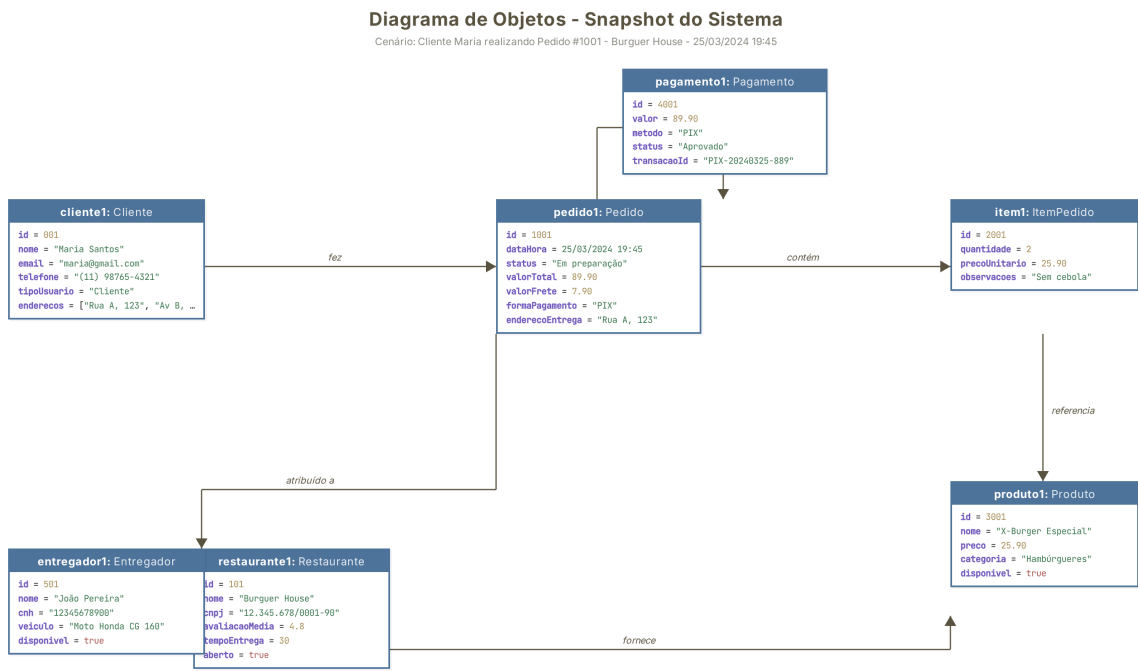


Figura 5.1 — Diagrama de objetos: cenário do Pedido #1001

5.3 Descrição das Instâncias

Objeto	Classe	Estado
cliente1	Cliente	Maria Santos, email maria@gmail.com, telefone (11) 98765-4321, 2 endereços cadastrados
restaurante1	Restaurante	Burguer House, CNPJ 12.345.678/0001-90, avaliação 4.8, tempo de entrega 30 min, aberto = true
pedido1	Pedido	ID 1001, status "Em preparação", valor total R\$ 89,90 (R\$ 82,00 itens + R\$ 7,90 frete), pago via PIX

Objeto	Classe	Estado
item1	ItemPedido	Quantidade 2, preço unitário R\$ 25,90, subtotal R\$ 51,80, observação "Sem cebola"
produto1	Produto	X-Burger Especial, categoria Hambúrgueres, preço R\$ 25,90, disponível = true
pagamento1	Pagamento	Valor R\$ 89,90, método PIX, status "Aprovado", transação PIX-20240325-889
entregador1	Entregador	João Pereira, CNH 12345678900, veículo Moto Honda CG 160, disponível = true

Tabela 5.1 — Instâncias representadas no diagrama de objetos

Os **links** entre objetos (representados por setas no diagrama) materializam as associações definidas no diagrama de classes: cliente1 fez pedido1, pedido1 contém item1, item1 referencia produto1, pedido1 pago com pagamento1, pedido1 atribuído a entregador1, e restaurante1 fornece produto1. Cada link possui um rótulo descritivo que esclarece a semântica da relação, facilitando a leitura por stakeholders não técnicos.

6. Diagrama de Pacotes

O diagrama de pacotes apresenta a **organização arquitetural** do sistema em camadas lógicas, agrupando classes e componentes relacionados em pacotes (namespaces ou módulos). É fundamental para visualizar dependências entre subsistemas e garantir que a arquitetura respeite princípios como Separação de Responsabilidades (SoC) e Baixo Acoplamento. A arquitetura aqui apresentada segue o padrão em camadas clássico, com separação clara entre apresentação, regras de negócio, persistência e integrações.

6.1 Visão Arquitetural

A arquitetura proposta divide o sistema em **sete pacotes** com responsabilidades distintas. As dependências seguem o fluxo descendente: a camada de Apresentação depende de Negócio, que depende de Dados e Integrações. As camadas transversais (Segurança, Utilitários e Configuração) são consumidas por todas as demais. Esta estrutura facilita a manutenção, permite substituição de componentes (ex.: troca de PostgreSQL por MySQL sem afetar a camada de negócio) e suporta deploy independente em arquitetura de microsserviços.

6.2 Diagrama de Pacotes

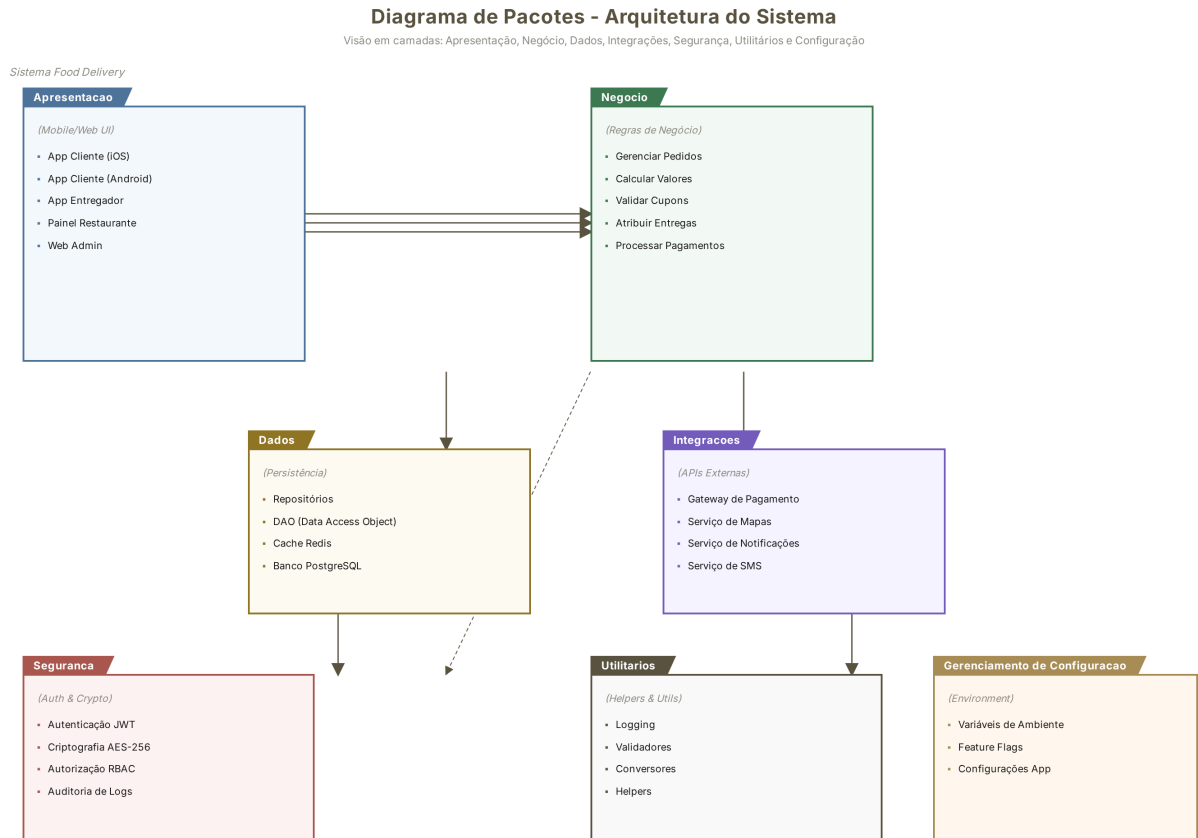


Figura 6.1 — Diagrama de pacotes do Sistema Food Delivery

6.3 Descrição dos Pacotes

Pacote	Camada	Conteúdo e Responsabilidade
Apresentacao	Frontend	Apps móveis (Cliente iOS/Android, Entregador Android) e painéis web (Restaurante e Admin). Contém componentes de UI, controllers de tela, gerenciadores de estado e clientes HTTP
Negocio	Backend Core	Serviços de domínio: PedidoService, PagamentoService, CupomService, EntregaService, AvaliacaoService. Implementa regras de negócio e orquestra chamadas entre Dados e Integrações
Dados	Persistência	Repositórios (Repository Pattern), objetos DAO, cache Redis para consultas frequentes e conexão com banco PostgreSQL principal
Integracoes	APIs Externas	Adapters para gateways de pagamento (Stripe, Mercado Pago), serviços de mapa (Google Maps, Mapbox), notificações push (Firebase FCM) e SMS (Twilio, TotalVoice)
Seguranca	Cross-cutting	Autenticação JWT, criptografia AES-256, autorização RBAC (Role-Based Access Control) e auditoria de logs para conformidade LGPD

Pacote	Camada	Conteúdo e Responsabilidade
Utilitarios	Cross-cutting	Helpers compartilhados: logging estruturado, validadores de CPF/CNPJ/CEP, conversores de DTO e formatadores de moeda/data
Config	Infraestrutura	Gerenciamento de variáveis de ambiente, feature flags para deploy progressivo e configurações específicas por ambiente (dev/staging/prod)

Tabela 6.1 — Descrição dos pacotes da arquitetura

7. Diagrama de Sequência

O diagrama de sequência captura a **visão dinâmica** do sistema, mostrando a ordem temporal das mensagens trocadas entre objetos durante a execução de um cenário específico. Diferente do diagrama de classes (estrutura estática) e do diagrama de objetos (snapshot), a sequência foca no **comportamento** ao longo do tempo. As mensagens podem ser síncronas (setas cheias) ou assíncronas (setas tracejadas), e cada lifeline (linha de vida) representa um participante ativo.

7.1 Cenário: Cliente Realiza Pedido com Sucesso

O cenário modelado representa o fluxo feliz (happy path) completo de um pedido bem-sucedido. Cinco participantes interagem: o **Cliente** (usuário final), o **App Mobile** (frontend), o **Servidor** (backend), o **Restaurante** (painel do parceiro) e o **Gateway de Pagamento** (sistema externo). O fluxo inclui seleção de produtos, envio do pedido, validação de disponibilidade, processamento de pagamento, confirmação do restaurante e notificação ao cliente.

7.2 Diagrama de Sequência

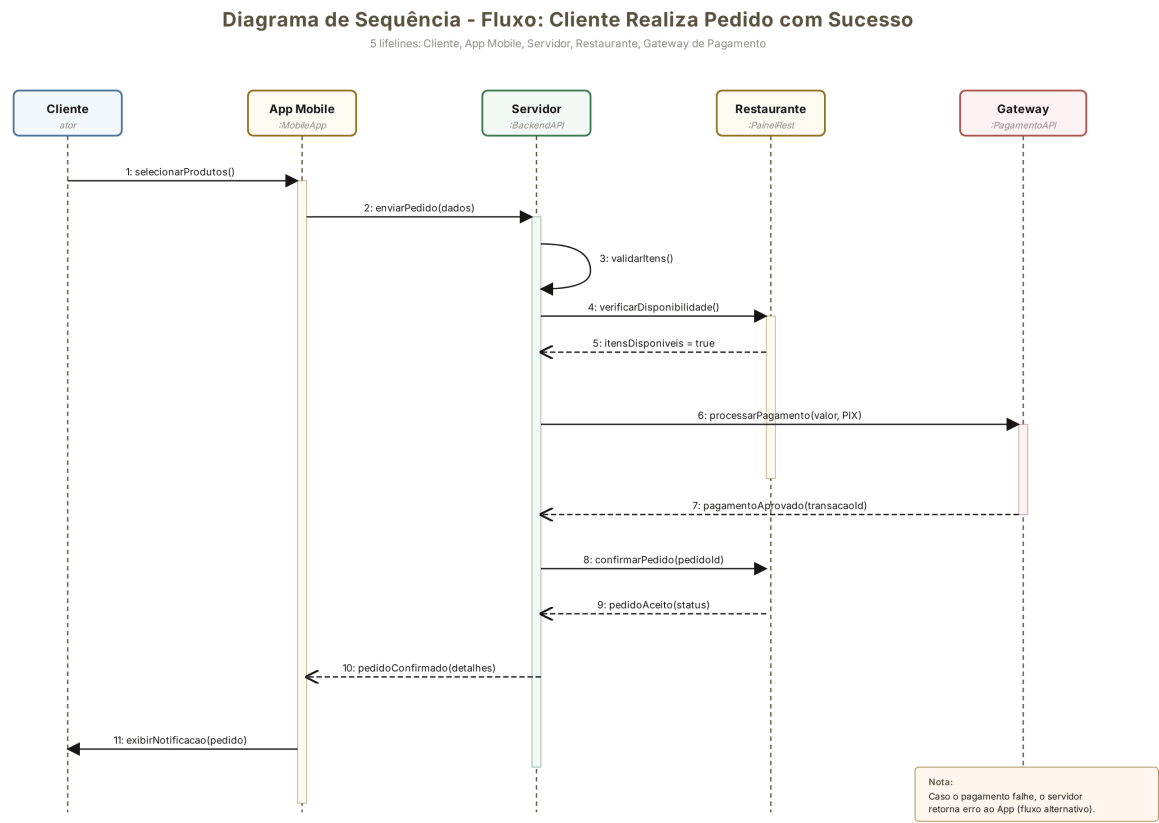


Figura 7.1 — Diagrama de sequência: fluxo de pedido com sucesso

7.3 Descrição Passo-a-Passo do Fluxo

Passo	De → Para	Mensagem	Tipo
1	Cliente → App	selecionarProdutos() — Cliente navega e adiciona itens ao carrinho	Síncrona
2	App → Servidor	enviarPedido(dados) — App submete o carrinho para o backend	Síncrona
3	Servidor → Servidor	validarItens() — Servidor valida estrutura e regras de negócio internamente	Self-call
4	Servidor → Restaurante	verificarDisponibilidade() — Consulta painel do restaurante sobre itens	Síncrona
5	Restaurante → Servidor	itensDisponiveis = true — Retorno confirmando disponibilidade	Assíncrona
6	Servidor → Gateway	processarPagamento(valor, PIX) — Solicita transação ao gateway externo	Síncrona
7	Gateway → Servidor	pagamentoAprovado(transacaoId) — Confirmação da transação PIX	Assíncrona

Passo	De → Para	Mensagem	Tipo
8	Servidor → Restaurante	confirmarPedido(pedidoId) — Notifica restaurante para iniciar preparo	Síncrona
9	Restaurante → Servidor	pedidoAceito(status) — Restaurante confirma e inicia preparação	Assíncrona
10	Servidor → App	pedidoConfirmado(detelhes) — Backend retorna sucesso ao app mobile	Assíncrona
11	App → Cliente	exibirNotificacao(pedido) — App exibe confirmação e dispara push notification	Síncrona

Tabela 7.1 — Detalhamento das mensagens do diagrama de sequência

Fluxo alternativo (não representado no diagrama): caso o pagamento seja recusado no passo 7, o Gateway retorna um erro em vez da confirmação. O Servidor então propaga o erro ao App, que oferece ao cliente a opção de tentar outro método de pagamento. O pedido permanece com status 'Aguardando Pagamento' e pode ser cancelado automaticamente após 10 minutos sem sucesso.

8. Exemplo de Implementação em Java

Esta seção apresenta um exemplo concreto de implementação em Java das classes **Pedido** e **Entregador**, demonstrando como os relacionamentos UML <> e <> se traduzem em código executável. O exemplo é didático e não inclui todos os detalhes de uma implementação de produção (tratamento de exceções robusto, validações de bean, injeção de dependência), mas evidencia o mapeamento direto entre conceitos de modelagem e construções da linguagem.

8.1 Classe Pedido

A classe **Pedido** encapsula o comportamento de um pedido, incluindo o método `fazerPedido()` que orquestra os casos de uso inclusos e estendidos. Note como os includes são chamadas obrigatórias (métodos privados chamados sempre), enquanto os extends são condicionais (chamados dentro de if).

```
public class Pedido {
    private int id;
    private DateTime dataHora;
    private String status;
    private double valorTotal;
    private double valorFrete;
    private String formaPagamento;
    private String enderecoEntrega;
    int tempoEstimado; // visibilidade package (~)

    private List<ItemPedido> itens;
    private Pagamento pagamento;
```

```
private Cliente cliente;
private Restaurante restaurante;
private Entregador entregador;

public void fazerPedido() {
    // Include: VerificarDisponibilidade (obrigatório)
    verificarDisponibilidade();

    // Include: RealizarPagamento (obrigatório)
    realizarPagamento();

    // Extend: AplicarCupomDesconto (opcional, condicional)
    if (temCupom()) {
        aplicarCupom();
    }

    // Extend: AgendarEntrega (opcional, condicional)
    if (entregaAgendada()) {
        agendarEntrega();
    }
}

public void rastrearPedido() {
    // Include: ConsultarLocalizacao (obrigatório)
    Localizacao loc = consultarLocalizacao();
    exibirNoMapa(loc);
}

public void cancelarPedido() {
    if (status.equals("Em preparação")) {
        // Include: SolicitarReembolso (obrigatório)
        pagamento.estornar();
        status = "Cancelado";
    }
}

private void verificarDisponibilidade() {
    for (ItemPedido item : itens) {
        if (!item.getProduto().isDisponivel()) {
            throw new RuntimeException("Produto indisponível: "
                + item.getProduto().getNome());
        }
    }
}
}
```

8.2 Classe Entregador

A classe **Entregador** herda de **Usuario** (herança) e implementa o comportamento de aceitar entregas e atualizar status. Observe como o método atualizarStatus() incorpora um extend (Reportar Problema, condicional) e um include (Confirmar Entrega, obrigatório em determinado estado).

```
public class Entregador extends Usuario { // Herança
    private String cnh;
    private String veiculo;
    private boolean disponivel;

    public void aceitarEntrega(Pedido pedido) {
```



```

    if (disponivel) {
        // Include: VerificarRota (obrigatório)
        Rota rota = verificarRota(pedido.getEnderecoEntrega());
        if (rota.isViavel()) {
            pedido.setEntregador(this);
            pedido.atualizarStatus("Saiu para entrega");
            disponivel = false;
        }
    }
}

public void atualizarStatus(Pedido pedido, String novoStatus) {
    pedido.setStatus(novoStatus);

    // Extend: ReportarProblema (opcional, condicional)
    if (novoStatus.equals("Problema")) {
        reportarProblema(pedido);
    }

    // Include: ConfirmarEntrega (obrigatório ao concluir)
    if (novoStatus.equals("Entregue")) {
        confirmarEntrega(pedido);
        disponivel = true;
    }
}

private Rota verificarRota(String endereco) {
    // Integração com serviço de mapas (Google Maps API)
    return mapaService.calcularRota(endereco);
}
}

```

8.3 Mapeamento UML → Código

A tabela abaixo sintetiza como cada conceito UML se manifesta no código Java. Este mapeamento ajuda desenvolvedores a traduzir diagramas em implementações consistentes e ajuda arquitetos a revisar se o código respeita o modelo.

Conceito UML	Construção Java	Exemplo no Código
Herança	<code>extends</code>	<code>public class Entregador extends Usuario</code>
Associação	Atributo + <code>getter/setter</code>	<code>private Cliente cliente; // em Pedido</code>
Composição	Atributo + criação no construtor	<code>private List itens = new ArrayList<>();</code>
Agregação	Atributo sem posse de ciclo de vida	<code>private Restaurante restaurante; // em Pedido</code>
<>	Chamada de método obrigatória	<code>verificarDisponibilidade(); // sempre chamado</code>
<>	Chamada condicional (if)	<code>if (temCupom()) { aplicarCupom(); }</code>
Visibilidade -	<code>private</code>	<code>private int id;</code>

Conceito UML	Construção Java	Exemplo no Código
Visibilidade #	<code>protected</code>	<code>protected String tipoUsuario; // em Usuario</code>
Visibilidade +	<code>public</code>	<code>public void fazerPedido()</code>
Visibilidade ~	<code>package-private</code>	<code>int tempoEstimado; // sem modificador</code>

Tabela 8.1 — Mapeamento de conceitos UML para construções Java

9. Tabela Resumo de Relações UML

A UML define oito tipos principais de relacionamentos que podem ocorrer entre elementos de um modelo. Compreender as diferenças entre cada um é essencial para produzir diagramas corretos e não ambíguos. A tabela a seguir apresenta cada relação com seu símbolo, um exemplo concreto do sistema de delivery e a justificativa para a escolha daquele tipo específico.

Relação	Símbolo	Exemplo no Food Delivery	Justificativa
Associação	----	Cliente → Pedido	Cliente conhece seus pedidos; relação estrutural entre objetos independentes
Herança	----▷	Cliente → Usuario	Cliente é um tipo de Usuario; herda atributos e comportamentos da superclasse
Dependência	---->	Controller → Service	Controller depende do Service em tempo de execução, mas não o possui como atributo
Agregação	◇----	Restaurante ◇---- Produto	Restaurante agrega produtos, mas produtos poderiam existir independentemente em catálogo genérico
Composição	◆----	Pedido ◆---- ItemPedido	ItemPedido não existe sem Pedido; ciclo de vida totalmente dependente (deleta em cascata)
Include	<>	Fazer Pedido → Realizar Pagamento	Pagamento é obrigatório e sempre executado quando um pedido é feito
Extend	<>	Fazer Pedido → Aplicar Cupom	Cupom é opcional, aplicado apenas sob condição (cliente possui cupom válido)
Realização	--▷	Pagamento implements IPagamento	Pagamento implementa o contrato definido pela interface IPagamento

Tabela 9.1 — Resumo dos relacionamentos UML aplicados ao sistema

A distinção mais sutil — e frequentemente fonte de erros em modelagem — é entre **agregação** e **composição**. Ambas representam relacionamentos todo-parte, mas a composição implica em posse exclusiva do ciclo de vida: se o todo é destruído, as partes também são. Na agregação, as partes podem sobreviver ao todo. No sistema de delivery, ItemPedido é composição de Pedido (não faz sentido existir um item sem pedido), enquanto Produto é agregação de Restaurante (um produto poderia ser transferido para outro restaurante ou existir em catálogo compartilhado).

10. Diferenciais do Sistema

Além das funcionalidades core modeladas nos diagramas anteriores, o sistema possui diferenciais importantes que o destacam em um mercado competitivo. Esta seção resume as funcionalidades por canal (mobile e web admin) e as integrações com serviços externos essenciais para a operação.

10.1 Funcionalidades Mobile (Android/iOS)

App	Plataforma	Funcionalidades-Chave
App do Cliente	iOS + Android	Busca e filtro de restaurantes, carrinho de compras, checkout multi-método, rastreamento em tempo real, avaliações, histórico, chat com entregador, gestão de endereços e cupons
App do Entregador	Android	Recebimento de notificações de entrega, aceitação/recusa, navegação GPS integrada, atualização de status, registro de problemas, controle de disponibilidade e ganhos
App do Restaurante	iOS + Android + Web	Gestão de pedidos em tempo real, aceitação/rejeição, atualização de cardápio, controle de disponibilidade de produtos, métricas de vendas e promoções

Tabela 10.1 — Funcionalidades dos aplicativos móveis

10.2 Funcionalidades Web Admin

Módulo	Funcionalidades
Dashboard	Métricas em tempo real: GMV (Gross Merchandise Volume), ticket médio, número de pedidos por hora, restaurantes ativos, NPS e taxa de cancelamento
Gestão de Usuários	Bloqueio e desbloqueio de clientes/entregadores, verificação de documentos (CPF, CNH, antecedentes), gestão de roles e permissões

Módulo	Funcionalidades
Gestão de Restaurantes	Aprovação de novos cadastros, gestão de comissões, suporte a disputes, suspensão temporária e encerramento de parcerias
Gestão Financeira	Comissões por pedido, repasses aos restaurantes, conciliação bancária, relatórios fiscais e exportação para sistemas contábeis
Marketing	Criação de campanhas segmentadas, cupons de desconto com regras (primeira compra, recorrente, frete grátis), push notifications em massa e relatórios de conversão

Tabela 10.2 — Funcionalidades do painel administrativo web

10.3 Integrações com Serviços Externos

A operação de um sistema de delivery depende criticamente de integrações com terceiros para pagamentos, mapas, notificações e comunicação. A escolha de fornecedores consolidados reduz risco operacional e acelera o time-to-market. A arquitetura deve isolar essas integrações por trás de interfaces (Adapter Pattern) para facilitar trocas futuras.

Categoria	Fornecedores	Caso de Uso
Pagamentos	Stripe, Mercado Pago, PayPal	Processamento de cartão de crédito, PIX, carteira digital; split de pagamento entre plataforma e restaurante
Mapas	Google Maps, Mapbox	Geocodificação de endereços, cálculo de rotas, estimativa de tempo de entrega, exibição de mapa em tempo real
Notificações	Firebase Cloud Messaging (FCM)	Push notifications para apps mobile (status de pedido, promoções, alertas de entrega)
SMS	Twilio, TotalVoice	SMS de confirmação de pedido, códigos de verificação por telefone, alertas críticos de falha
Storage	AWS S3, Cloudflare R2	Armazenamento de fotos de produtos, avatares de usuários, documentos de validação (CNH, comprovante)
Observabilidade	Datadog, Sentry, New Relic	Monitoramento de performance, erros em produção, tracing distribuído e alertas em tempo real

Tabela 10.3 — Integrações com serviços externos

11. Considerações Finais

Este documento apresentou um projeto completo de modelagem UML para um sistema de delivery de comida, cobrindo desde o levantamento de requisitos até exemplos de implementação. A modelagem formal — muitas vezes subestimada em favor de abordagens ágeis puras — demonstra seu valor em sistemas de complexidade média a alta como o food delivery, onde múltiplos atores, transações financeiras, geolocalização em tempo real e regras de negócio intrincadas precisam coexistir sem ambiguidades.

Os artefatos produzidos (15 requisitos funcionais, 12 não funcionais, diagramas de casos de uso, classes, objetos, pacotes e sequência) formam um conjunto coeso que pode ser apresentado a diferentes stakeholders: investidores (casos de uso e requisitos), desenvolvedores (classes e sequência), arquitetos (pacotes) e QA (descrições textuais de casos de uso com fluxos alternativos). Esta versatilidade é uma das principais vantagens da UML quando bem aplicada.

Como próximos passos do projeto, recomenda-se: (i) **detalhamento de diagramas complementares** — diagrama de atividades para fluxos complexos como cancelamento, diagrama de máquina de estados para o ciclo de vida do Pedido, e diagrama de componentes para visão de deploy; (ii) **prototipação de baixa fidelidade** das telas principais (busca, carrinho, rastreamento) para validação com usuários antes da implementação; (iii) **definição da API REST** com OpenAPI/Swagger derivada do modelo de classes; (iv) **planejamento de testes** baseado nas descrições de casos de uso, mapeando cada fluxo principal e alternativo para casos de teste automatizados; e (v) **setup de infraestrutura** com CI/CD, ambientes (dev/staging/prod) e monitoramento desde o primeiro sprint.

A lição central deste projeto é que **modelagem e implementação não são etapas concorrentes, mas complementares**. A UML não substitui código — ela reduz o desperdício de retrabalho, alinha expectativas e preserva o conhecimento do domínio ao longo do ciclo de vida do software. Em uma startup em crescimento com 500+ restaurantes, onde a equipe tende a dobrar a cada semestre, este tipo de documentação é o que permite que novos desenvolvedores sejam produtivos em semanas e não em meses.